Advanced Machine learning Mastering Course

Introduced by

George Samuel

Master in computer science
Cairo University

Innovisionray.com

2024

Advanced Machine Learning 2024 by George Samuel

Machine Learning Diploma

1- Statistics
2- Pandas

1. Pandas Basics

Import:

```
1 import pandas as pd
```

Create Series:

With default index

```
1 s = pd.Series([1, 2, 3, 4])
2 s
```

```
0    1
1    2
2    3
3    4
dtype: int64
```

Specify the index

```
1 # Specify the indeces
2 s = pd.Series([1, 2, 3, 4], index = ["A", "B", "C", "D"])
3 s
```

```
A    1
B    2
C    3
D    4
dtype: int64
```

Create DataFrame:

```
data = [[1, 444, 'abc'],
        [2, 555, 'def'],
        [3, 666, 'ghi'],
        [4, 444, 'xyz']]
df = pd.DataFrame(data, columns=["col1", "col2", "col3"])
df
```

	col1	col2	col3
0	1	444	abc
1	2	555	def
2	3	666	ghi
3	4	444	xyz

```
# another way
data = {'col1':[1,2,3,4],
        'col2':[444,555,666,444],
        'col3':['abc','def','ghi','xyz']}

df = pd.DataFrame(data)
df
```

	col1	col2	col3
0	1	444	abc
1	2	555	def
2	3	666	ghi
3	4	444	xyz

Rename DataFrame Columns & index:

Rename DataFrame Columns

```
df = pd.DataFrame([[1, 444, 'abc'],
                   [2, 555, 'def'],
                   [3, 666, 'ghi'],
                   [4, 444, 'xyz']])
display(df)
columns=["col1", "col2", "col3"]
df.columns = columns
display(df)
```

	0	1	2
0	1	444	abc
1	2	555	def
2	3	666	ghi
3	4	444	xyz

	col1	col2	col3
0	1	444	abc
1	2	555	def
2	3	666	ghi
3	4	444	xyz

Rename DataFrame index

```
df = pd.DataFrame([[1, 444, 'abc'],
                   [2, 555, 'def'],
                   [3, 666, 'ghi'],
                   [4, 444, 'xyz']])
display(df)
index = ["row1", "row2", "row3", "row4"]
df.index = index
display(df)
```

	0	1	2
0	1	444	abc
1	2	555	def
2	3	666	ghi
3	4	444	xyz

	0	1	2
row1	1	444	abc
row2	2	555	def
row3	3	666	ghi
row4	4	444	xyz

Pandas Dtypes:

Bool ➢ Represents Numerical datatypes with True & False values.	**Int** ➢ Represents Numerical datatypes with integer values.	**Float** ➢ Represents Numerical datatypes with continuous values.
Category ➢ Represents Categorical datatypes.	**Object** ➢ Is a mix of categorical datatypes & Numerical datatypes. ➢ Can carry any python object; such as, lists, tuples, strings, etc.	

Pandas Dtypes:

Get Datatypes of all columns

```
1  # Datatype of all columns
2  df.dtypes
```

```
col1      int64
col2      int64
col3     object
dtype: object
```

Get Datatype of one column

```
1  # Datatype of one column
2  df['col1'].dtype
```

```
dtype('int64')
```

Change Datatype:

Change Datatype of one column

```
df["col1"] = df["col1"].astype("category")
df.dtypes
```

```
col1    category
col2       int64
col3      object
dtype: object
```

Change Datatypes of group of columns

```
cols = ["col1", "col3"]
df[cols] = df[cols].astype("category")
df.dtypes
```

```
col1    category
col2       int64
col3    category
dtype: object
```

2. EDA using Pandas

EDA using Pandas:

- EDA is about exploring and understanding the data and getting insights about it.
- EDA is short for Exploratory Data Analysis.
- Pandas provides built-in methods and features that helps us to answer different questions about the data.

EDA using Pandas:

Get the first n rows of DataFrame

```
1 df.head(2)
```

	col1	col2	col3
0	1	444	abc
1	2	555	def

Get the last n rows of DataFrame

```
1 df.tail(2)
```

	col1	col2	col3
2	3	666	ghi
3	4	444	xyz

Get Statistical Measures about Numerical Columns

```
1 df.describe()
```

	count	mean	std	min	25%	50%	75%	max
col2	4.0	527.25	106.274409	444.0	444.0	499.5	582.75	666.0

Get Statistical Measures about Categorical Columns

```
1 df.describe(include="category")
```

	col1
count	4
unique	4
top	1
freq	1

EDA using Pandas:

Get DataFrame Rows Names

```
1 df.index
```

```
RangeIndex(start=0, stop=4, step=1)
```

Get DataFrame Columns Names

```
1 df.columns
```

```
Index(['col1', 'col2', 'col3'], dtype='object')
```

Get Unique Values

```
1 # get the unique values
2 df['col2'].unique()
```

```
array([444, 555, 666], dtype=int64)
```

Get Unique Values Number

```
1 # get number of unique values
2 df['col2'].nunique()
```

```
3
```

EDA using Pandas:

Get Max Value

```
1 df["col2"].max()
```

```
666
```

Get Element whose value is Max

```
1 df.col2.idxmax()
```

```
2
```

Get Min Value

```
1 df["col2"].min()
```

```
444
```

Get Element whose value is Min

```
1 df.col2.idxmin()
```

```
0
```

EDA using Pandas:

DataFrame Basic Information

```
1 df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4 entries, 0 to 3
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   col1    4 non-null      category
 1   col2    4 non-null      int64
 2   col3    4 non-null      object
dtypes: category(1), int64(1), object(1)
memory usage: 400.0+ bytes
```

Unique Values Frequency

```
1 d = df['col2'].value_counts()
2 d
```

```
444    2
555    1
666    1
Name: col2, dtype: int64
```

Get Max Value of All Columns

```
1 df.max()
```

```
col2      666
col3      xyz
dtype: object
```

Summation

```
1 df['col2'].sum()
```

```
2109
```

3. Data Manipulation

Data Manipulation:

- Pandas provides built-in methods and attributes that allows you to apply different operations over Pandas DataFrames or Series.
- Below are the most popular & Important attributes that allows you to apply data manipulation.

Data Manipulation (Pandas to Numpy):

Convert Pandas Series into Numpy 1D-Array

```
1  df['col1'].values
```

```
[1, 2, 3, 4]
Categories (4, int64): [1, 2, 3, 4]
```

Convert Pandas DataFrame into Numpy 2D-Array

```
1  df.values
```

```
array([[1, 444, 'abc'],
       [2, 555, 'def'],
       [3, 666, 'ghi'],
       [4, 444, 'xyz']], dtype=object)
```

Data Manipulation (Replace Values):

Replace a Single Value

```
1 df.replace(555, "ali")
```

	col1	col2	col3
0	1	444	abc
1	2	ali	def
2	3	666	ghi
3	4	444	xyz

Replace Multiple Values using Dictionary

```
1 df.replace({444: "omar", "abc": 444})
```

	col1	col2	col3
0	1	omar	444
1	2	555	def
2	3	666	ghi
3	4	omar	xyz

Replace Multiple Values by One Value

```
1 df.replace([1, 666, "abc"], "aaa")
```

	col1	col2	col3
0	aaa	444	aaa
1	2	555	def
2	3	aaa	ghi
3	4	444	xyz

Data Manipulation (Mapping):

Before Mapping

```
1 df
```

	col1	col2	col3
0	1	444	abc
3	4	444	xyz
1	2	555	def
2	3	666	ghi

After Mapping

```
1 df.col2 = df.col2.map({444: "Fours", 555: "Fives", 666: "Sixs"})
2 df
```

	col1	col2	col3
0	1	Fours	abc
3	4	Fours	xyz
1	2	Fives	def
2	3	Sixs	ghi

Data Manipulation (Sorting):

Ascending sorting

```
1 sorted_df = df.sort_values(by='col1')
2 sorted_df
```

	col1	col2	col3
0	1	Fours	abc
1	2	Fives	def
2	3	Sixs	ghi
3	4	Fours	xyz

Descending sorting

```
1 sorted_df = df.sort_values(by='col1', ascending=False)
2 sorted_df
```

	col1	col2	col3
3	4	Fours	xyz
2	3	Sixs	ghi
1	2	Fives	def
0	1	Fours	abc

Data Manipulation (Apply method):

- Is a methods used to apply a certain function to each sample in the DataFrame.

Example1

```
1  def duplicate(x):
2      return x*2
3
4  df['col1'].apply(duplicate)
```

```
0    11
1    22
2    33
3    44
Name: col1, dtype: object
```

Example2

```
1  # apply built in function
2  df['col1'].apply(len)
```

```
0    1
1    1
2    1
3    1
Name: col1, dtype: int64
```

Example3

```
1  # or use lambda function
2  df['col1'].apply(lambda x: x*2)
```

```
0    11
1    22
2    33
3    44
Name: col1, dtype: object
```

Example4

```
1  df2 = pd.DataFrame([[1, 'ALI'   , 3],
2                      [5, 'OMAR' , 8],
3                      [4, 'AHMED', 9]])
4  df2.apply(lambda x: x*2)
```

	0	1	2
0	2	ALIALI	6
1	10	OMAROMAR	16
2	8	AHMEDAHMED	18

4. Indexing & Slicing

Indexing:

- Means accessing one element in Series or DataFrame, using its index or name.
- There are two ways to apply indexing:
  - Either using the element's Index.
  - Or using the element's Name.

Slicing:

- Means accessing many elements in Series or DataFrame, by specifying a range of Indices or name.
- There are two ways to apply Slicing:
  - Either, using a range of elements' Indices.
  - Or using a range of elements' Names.

Indexing & Slicing using Index:

- **Series Indexing**

```
1 df.col1.iloc[2]
```

```
3
```

- **Matrix Indexing**

```
1 df.iloc[2, 1]
```

```
666
```

- **Series Slicing**

```
1 df.col1.iloc[0:2]
```

```
0    1
1    2
Name: col1, dtype: category
Categories (4, int64): [1, 2, 3, 4]
```

- **Matrix Slicing**

```
1 df.iloc[:, 0:2]
```

	col1	col2
0	1	444
1	2	555
2	3	666
3	4	444

Indexing & Slicing using Names:

- **Series Indexing**

```
1 df.col1.loc[3]
```

```
4
```

- **Matrix Indexing**

```
1 df.loc[2, "col1"]
```

```
3
```

- **Series Slicing**

```
1 df.col1.loc[2:3]
```

```
2    3
3    4
Name: col1, dtype: category
Categories (4, int64): [1, 2, 3, 4]
```

- **Matrix Slicing**

```
1 df.loc[:, "col1":"col2"]
```

	col1	col2
0	1	444
1	2	555
2	3	666
3	4	444

5. Inserting/Dropping DataFrame Columns & Rows

Insert New Columns:

The first way

```
1  new_col = df.col1 + df.col2
2  df.insert(3,"new" , new_col)
3  df
```

	col1	col2	col3	new
0	1	444	abc	445
1	2	555	def	557
2	3	666	ghi	669
3	4	444	xyz	448

Another way

```
1  df['new'] = df.col1 + df.col2
2  df
```

	col1	col2	col3	new
0	1	444	abc	445
1	2	555	def	557
2	3	666	ghi	669
3	4	444	xyz	448

Drop Columns:

Drop one columns

```
1 df.drop('new',axis=1)
```

	col1	col2	col3
0	1	444	abc
1	2	555	def
2	3	666	ghi
3	4	444	xyz

Drop many columns

```
1 df.drop(['col1', 'new'],axis=1)
```

	col2	col3
0	444	abc
1	555	def
2	666	ghi
3	444	xyz

Insert New Rows:

```
1 new_row = {"col1": -1, "col2": 222, "col3": "OuO"}
2 df.append(new_row, ignore_index=True)
```

	col1	col2	col3
0	1	444	abc
1	2	555	def
2	3	666	ghi
3	4	444	xyz
4	-1	222	OuO

Drop Rows:

Drop one row

```
1 df.drop(2, axis=0)
```

	col1	col2	col3
0	1	444	abc
1	2	555	def
3	4	444	xyz

Drop many rows

```
1 df.drop([1, 3], axis=0)
```

	col1	col2	col3
0	1	444	abc
2	3	666	ghi

6. Null Values

What are Null Values?

- Null Values means missing values, which means that an element doesn't have a value, or have a value of None or Nan.
- Null Values occur due to problems during gathering data, for example a client forgot or to enter his age.

Check for Null Values:

```
1  null = df.isnull()
2  pd.DataFrame(null.sum()).T
```

	col1	col2	col3
0	0	1	2

Handle Null Values:

- There are three options to do to handle missing values:

1. Drop rows that contain Null Values.
2. Drop columns that contain Null Values.
3. Replace Null Values with Mean, Median, or Mode.

Handle Null Values (Drop Rows):

Drop all rows

```
1 df.dropna()
```

	col1	col2	col3
2	1	2.0	3.0

Drop rows in specific columns

```
1 df.dropna(subset=["col2"])
```

	col1	col2	col3
0	1	2.0	3.0
2	1	2.0	3.0

Handle Null Values (Drop Columns):

Drop all columns

```
1 df.dropna(axis=1)
```

	col1	col3
0	1	3.0
1	5	3.0
2	1	3.0

Drop Specific Columns

```
1 df.drop(["col3"], axis=1)
```

	col1	col2
0	1	2.0
1	5	NaN
2	1	2.0

Handle Null Values (Replace Rows Null Values):

```
1 mean = df.col3.mean()
2 df.col3 = df.col3.fillna(value=mean)
3 df
```

	col1	col2	col3
0	1	2.0	3.0
1	5	NaN	3.0
2	1	2.0	3.0

Assignment 1: The automated_stat_analyzer Function

A retail company needs a utility to quickly summarize sales data. Students must create a function that identifies the "Central Tendency" and "Dispersion" of any numerical column.

Requirements:

- Accept a Pandas DataFrame and a column name.
- Calculate the **Mean** ,**Median** dna ,**Standard Deviation**.
- Identify if the data is "Skewed" by comparing the Mean and Median.
- **Bonus** eht nruter ,lacirogetac si nmuloc eht fI :**Mode** instead.

```python
import pandas as pd
import numpy as np

data = {
    'Transaction_ID': range(1, 11),
    'Product_Category': ['Electronics', 'Home', 'Electronics', 'Sports', 'Home',
                         'Electronics', 'Home', 'Sports', 'Electronics', 'Electronics'],
    'Sales_Amount': [150, 200, 155, 300, 210, 180, 205, 1000, 190, 160], # 1000 is an Outlier
    'Customer_Age': [25, 34, np.nan, 45, 23, 31, 29, np.nan, 38, 40],    # Contains Nulls (NaN)
    'Rating': [5, 4, 3, 5, 2, 4, 5, 2, 4, 3]
}
```

```python
import pandas as pd

def automated_stat_analyzer(df, column_name):
    """
    Company Task: Provide a summary report of a specific data variable.

    Instructions:
    1. Check if the column is numerical or categorical.
    2. For numerical: Calculate Mean, Median, and Standard Deviation.
    3. For categorical: Calculate the Mode.
    4. Return a dictionary with these statistical measures.
    """
    # TODO: Implement using df[column_name].mean(), .median(), .std(), or .mode()
    pass
```

Assignment 1 The null_handling_strategy Function

Incoming user data often has missing values. Students must implement a flexible strategy to handle these "Null Values" to prepare data for Machine Learning

Requirements:

- Check for null values in the DataFrame.
- Apply a strategy based on parameters: "naidem_llif" ro ,"naem_llif" ,"swor_pord"
- Ensure the function only fills numerical columns when using mean or median.

```
import pandas as pd
import numpy as np

data = {
    'Transaction_ID': range(1, 11),
    'Product_Category': ['Electronics', 'Home', 'Electronics', 'Sports', 'Home',
                         'Electronics', 'Home', 'Sports', 'Electronics', 'Electronics'],
    'Sales_Amount': [150, 200, 155, 300, 210, 180, 205, 1000, 190, 160], # 1000 is an Outlier
    'Customer_Age': [25, 34, np.nan, 45, 23, 31, 29, np.nan, 38, 40],    # Contains Nulls (NaN)
    'Rating': [5, 4, 3, 5, 2, 4, 5, 2, 4, 3]
}
```

```
def null_handling_strategy(df, strategy="fill_mean"):
    """
    Company Task: Clean a dataset by resolving missing (NaN) values.
    """
    # TODO: Implement using .isnull(), .dropna(), or .fillna()
    pass
```

Thank You